

ML ACADEMY

ML / AI System Design Interview Handbook

Detailed Questions, Explanations & Worked Examples for Top Tech
Interviews

Requirements • Data • Features • Training • Serving • Monitoring Recommendations • Fraud •
Search • LLM Systems • Scale • Reliability

www.learnmlacademy.com

Free Resource - Share with your network

How to get the most from this handbook

This PDF is designed for serious interview preparation. Each topic combines an interview-ready answer with deeper explanation, examples, diagrams, code, calculations or system trade-offs wherever they improve understanding.

Recommended approach: answer the question yourself first, then compare with the explanation and work through the example.

For coding and SQL questions, state assumptions, approach, complexity and edge cases before writing code. For ML/AI system design, discuss requirements, trade-offs, failure modes, monitoring and security instead of claiming one architecture is always best.

Section	Topics
1. Research basis and further reading	1
TOTAL COVERAGE	1

1. The ML/AI system-design interview framework

The framework is a checklist, not a script. Spend the first minutes clarifying the system before selecting models.

A reusable 14-part framework 1) user decision, 2) business metric, 3) scale/SLO, 4) data, 5) labels, 6) baseline, 7) features/retrieval, 8) model, 9) offline/online paths, 10) deployment, 11) monitoring, 12) failure handling, 13) security/privacy, 14) trade-offs. Do not draw boxes before clarifying the decision and constraints.

Question to ask first	Why it changes the design
What decision is the system making?	Defines label, output, latency path and product metric.
What is the traffic/data scale?	Determines storage, candidate-generation strategy, caching and serving capacity.
What is the latency SLO?	Determines which features/models can run synchronously.
When do labels arrive?	Controls evaluation, retraining speed and experiment design.
What happens when the model/service fails?	Forces a safe fallback rather than a perfect-demo architecture.

2. Recommendation, search and ranking systems

These cases all use multi-stage candidate generation plus richer ranking because the full universe is too large to score expensively online.

SYSTEM DESIGN CASE2026 SIGNAL

Recent Meta MLE reports include TikTok-style recommendation design and offline/online evaluation.

Short-video recommendation feed

Requirements

Rank a personalized feed from millions of videos under ~150 ms p95 while balancing engagement, freshness, diversity and safety.

Architecture

Event stream feeds offline/nearline user-item features. Online, multiple candidate generators retrieve followed, collaborative, content-similar and trending videos. Feature service hydrates candidates; a learned ranker scores a shortlist; a policy/diversity layer enforces freshness, repetition and safety; results and impressions are logged for future training.

Success metrics

Offline recall@k/NDCG/calibration; online watch time, completion, return rate, diversity, creator distribution, complaint/safety rate and p95 latency.

Capacity / bottleneck

Candidate generation and feature hydration must fit inside the latency budget. Use multi-stage ranking, cache reusable embeddings/features and spend the heaviest model only on a small candidate set.

Failure modes

Popularity feedback loop, position bias, stale features, cold start, filter bubbles, unsafe content, duplicated videos, delayed labels.

Trade-offs

Quality vs latency; short-term engagement vs long-term retention/diversity; exploration vs immediate reward; freshness vs confidence.

Likely follow-up

How would you evaluate offline improvements before running an online experiment? How do you prevent the model from learning only exposure bias?

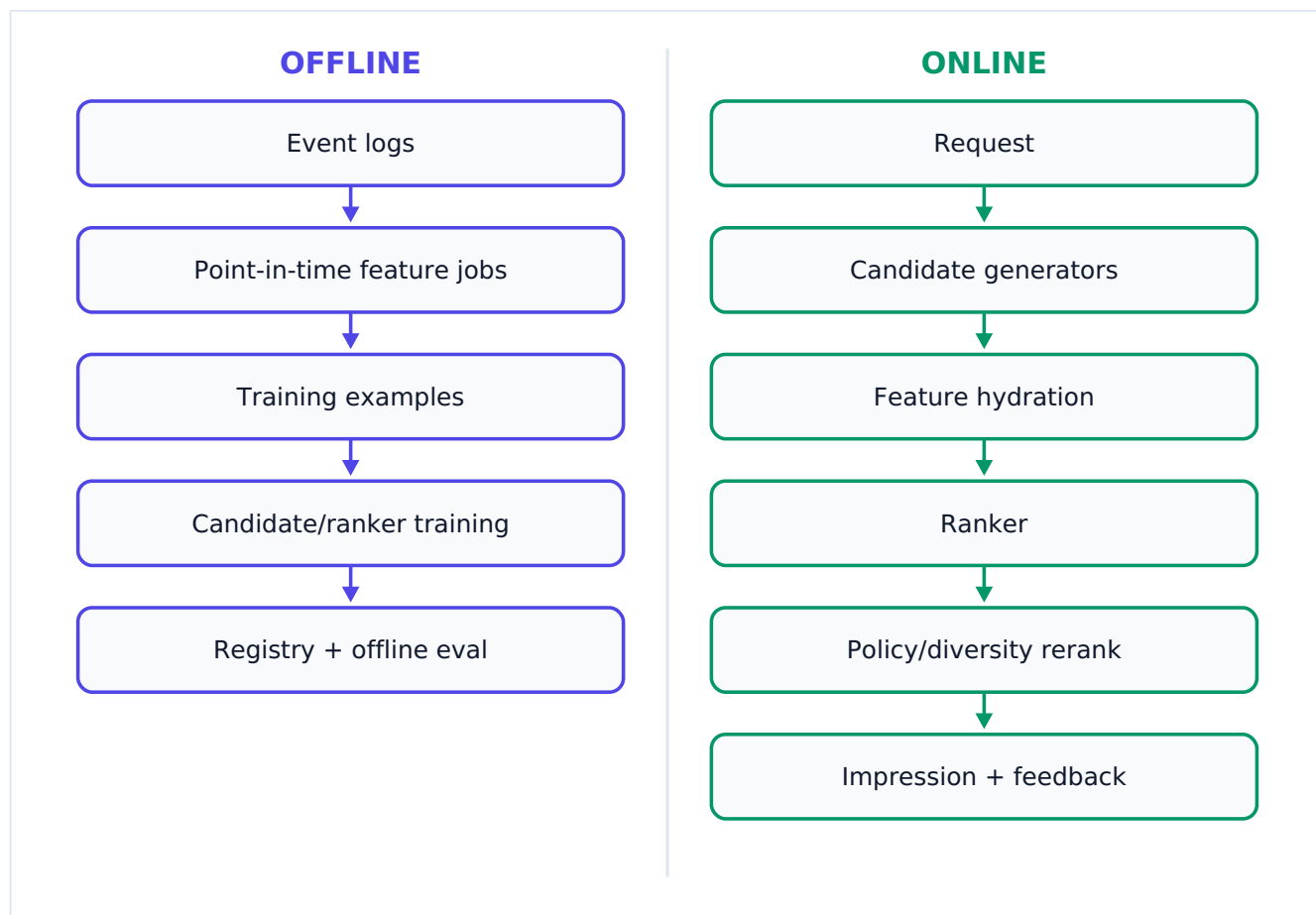


Figure: interview-level architecture. Explain arrows, failure boundaries and trade-offs.

SYSTEM DESIGN CASE2026 SIGNAL

Autocomplete/type-ahead remains a common interview design because it combines search, ranking, caching and product safety.

Search autocomplete / type-ahead

Requirements

Return useful suggestions within tens of milliseconds as the user types; support misspellings, trending queries, policy filtering and optional personalization.

Architecture

Use a compact prefix structure (trie/FST or search index) for fast candidate generation, normalize/spell-correct, mix popularity/trending/context candidates, then run a lightweight ranker. Cache hot prefixes and apply safety/policy filters. Personalization uses coarse, privacy-aware context rather than a heavyweight model on every keystroke.

Success metrics

MRR, suggestion CTR, keystrokes saved, zero-suggestion rate, inappropriate-suggestion rate, freshness and p95 latency.

Capacity / bottleneck

Every keystroke can issue a request. Prefix lookup and cache must be extremely cheap; personalized reranking must stay within a tiny budget.

Failure modes

Stale trends, offensive suggestions, private-query leakage, head-query domination, misspellings, query spam.

Trade-offs

Freshness vs cacheability; personalization vs privacy; neural ranking quality vs latency; aggressive correction vs user intent.

Likely follow-up

How would you handle a brand-new query with no historical clicks?

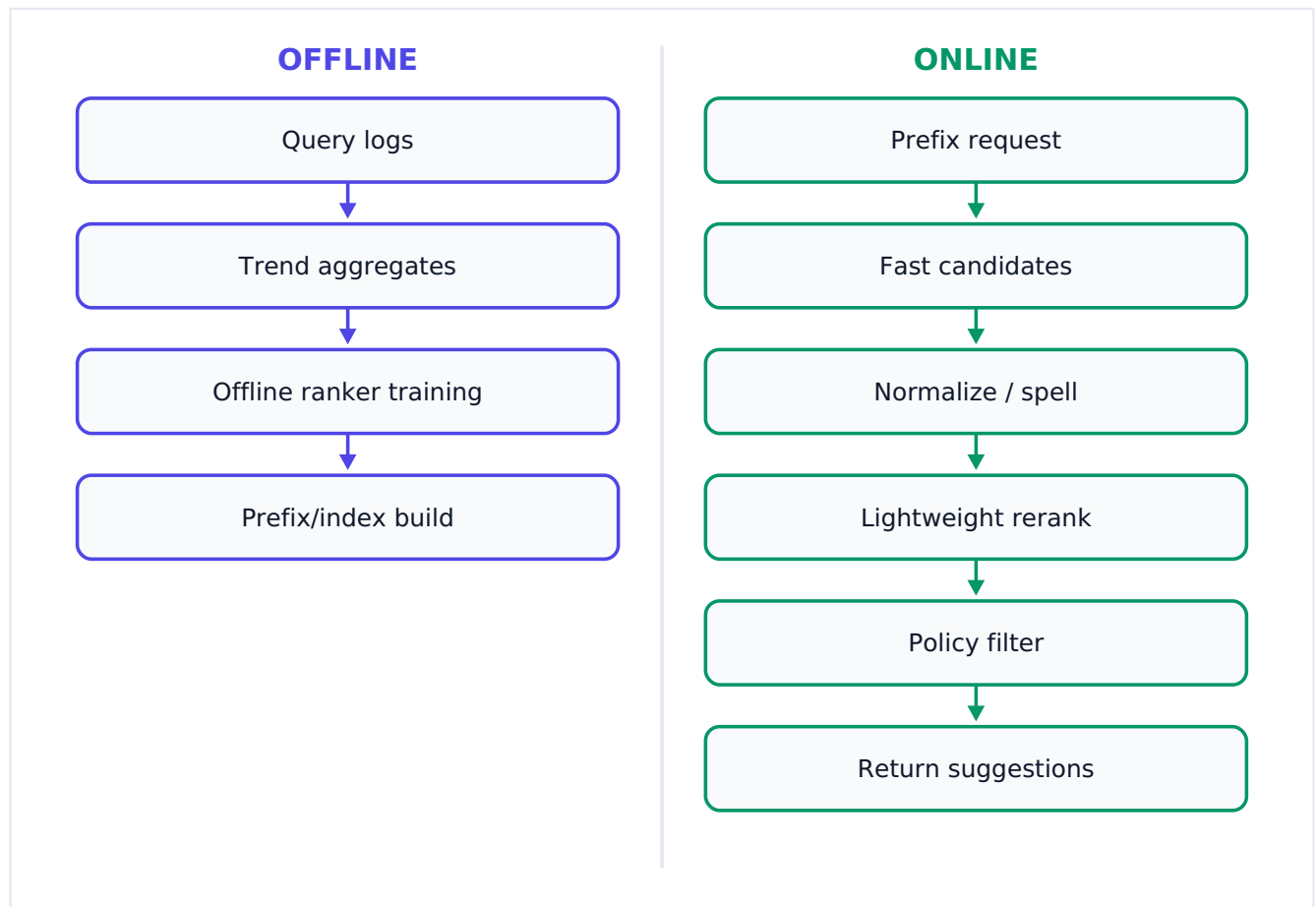


Figure: interview-level architecture. Explain arrows, failure boundaries and trade-offs.

SYSTEM DESIGN CASE2026 SIGNAL

Search and ranking systems are recurring Google/Meta-style ML system-design patterns.

Two-stage search ranking

Requirements

Search hundreds of millions of documents/items with high recall and useful top results under a 200 ms p95 budget.

Architecture

Query understanding feeds lexical and vector candidate retrieval with metadata/security filters. Merge candidates, hydrate features, run a learned reranker on a few hundred items, then apply diversity/business rules. Log query, impression, clicks and satisfaction signals with position information.

Success metrics

Retrieval recall@k; ranking NDCG/MRR; zero-result rate; click/satisfaction metrics; freshness; fairness; p95 latency.

Capacity / bottleneck

Full-corpus learned scoring is infeasible online, so cheap retrieval must narrow the universe dramatically before expensive ranking.

Failure modes

Vocabulary mismatch, stale index, unauthorized results, popularity bias, query drift, position-biased labels, duplicate near-identical results.

Trade-offs

Recall vs retrieval cost; neural complexity vs latency; freshness vs indexing cost; personalization vs privacy.

Likely follow-up

How would you learn from clicks without simply learning position bias?

SYSTEM DESIGN CASE2026 SIGNAL

Current MLE question banks include ads ranking and evaluation because it tests calibration, delayed labels and multi-objective trade-offs.

Ads ranking and marketplace evaluation

Requirements

Rank eligible ads for each impression while balancing user experience, advertiser outcomes, policy constraints, budgets and marketplace revenue.

Architecture

Eligibility/policy filters produce candidates. Models estimate CTR, conversion/value and quality. A calibrated value/auction layer combines bids and predicted outcomes, then frequency/diversity controls and pacing rules apply. Log impressions, clicks, conversions and auction context for delayed-label training.

Success metrics

Offline AUC/calibration/lift; online revenue, conversions, advertiser ROI, user engagement/complaints, policy violations and latency.

Capacity / bottleneck

High QPS under strict auction latency; expensive features and models need staged scoring and caching. Conversion labels are delayed and attribution is imperfect.

Failure modes

Click bias, delayed conversion, budget-pacing errors, cold-start advertisers, policy violations, marketplace feedback loops.

Trade-offs

Revenue vs user trust; exploration vs advertiser predictability; quality vs latency; global model vs market-specific behavior.

Likely follow-up

How would you design an offline + online evaluation framework for a new ads ranker?

3. Fraud, RAG, GPU serving and feature platforms

Senior cases test boundaries between data, models and systems: point-in-time correctness, authorization, serving capacity and fallbacks.

SYSTEM DESIGN CASE

Real-time fraud detection

Requirements

Score payment attempts around 2,000 requests/s with ~100 ms p95 and support approve/review/decline actions.

Architecture

Validated request -> idempotency check -> online point-in-time features -> model score -> calibrated threshold/rules -> approve, manual review or decline. Offline path builds leakage-safe labels and features, trains candidates, validates by time/cohort, then deploys canary versions. Log decisions, features, model/threshold versions and delayed outcomes.

Success metrics

PR-AUC or recall at fixed precision, fraud dollars prevented, false-decline rate, manual-review volume, calibration, p95 latency and cost.

Capacity / bottleneck

Fresh online features and external device/account signals must arrive within the synchronous budget; use fallbacks for missing features.

Failure modes

Target leakage, stale features, adversarial adaptation, feedback from blocked transactions, delayed/incorrect labels, review overload.

Trade-offs

Fraud capture vs customer friction; complex model vs latency; threshold aggressiveness vs review capacity.

Likely follow-up

How would you choose the operating threshold if reviewers can inspect only 5,000 cases/day?

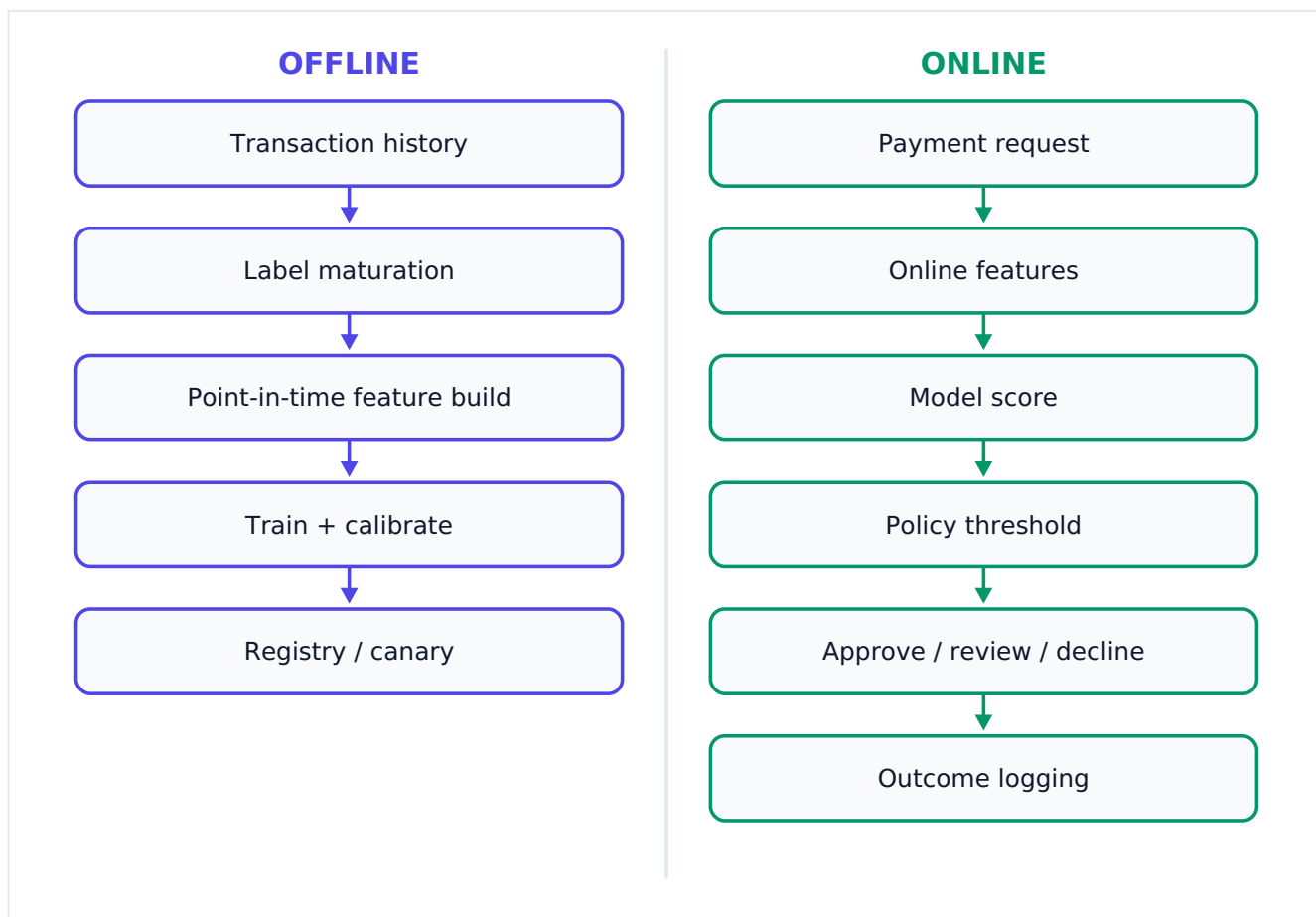


Figure: interview-level architecture. Explain arrows, failure boundaries and trade-offs.

SYSTEM DESIGN CASE2026 SIGNAL

Private-data RAG is a current AI-engineering system-design theme and exposes retrieval, security, evaluation and LLM trade-offs.

Enterprise private-data RAG

Requirements

Answer employee questions from internal documents with citations, tenant/document access control and reliable abstention when evidence is insufficient.

Architecture

Versioned ingestion parses/chunks documents and attaches source + ACL metadata. Online, identity creates authorization filters, hybrid retrieval collects candidates, reranker selects evidence, context builder keeps provenance, LLM generates, then citation/claim checks validate support. All stages are traced.

Success metrics

Retrieval recall@k, authorized-retrieval rate, grounded/citation support, answer correctness, abstention accuracy, freshness, p95 latency and cost.

Capacity / bottleneck

Reranking and generation consume latency; context budget is scarce. Cache only where tenant/user permissions permit.

Failure modes

Access leak, stale index, prompt injection in documents, unsupported claim, missing evidence, conflicting policy versions.

Trade-offs

Recall vs context precision; model size vs latency/cost; freshness vs indexing churn; strict refusal vs user coverage.

Likely follow-up

How do you distinguish retrieval failure from generation failure? What if the vector DB cannot express complex ACLs?

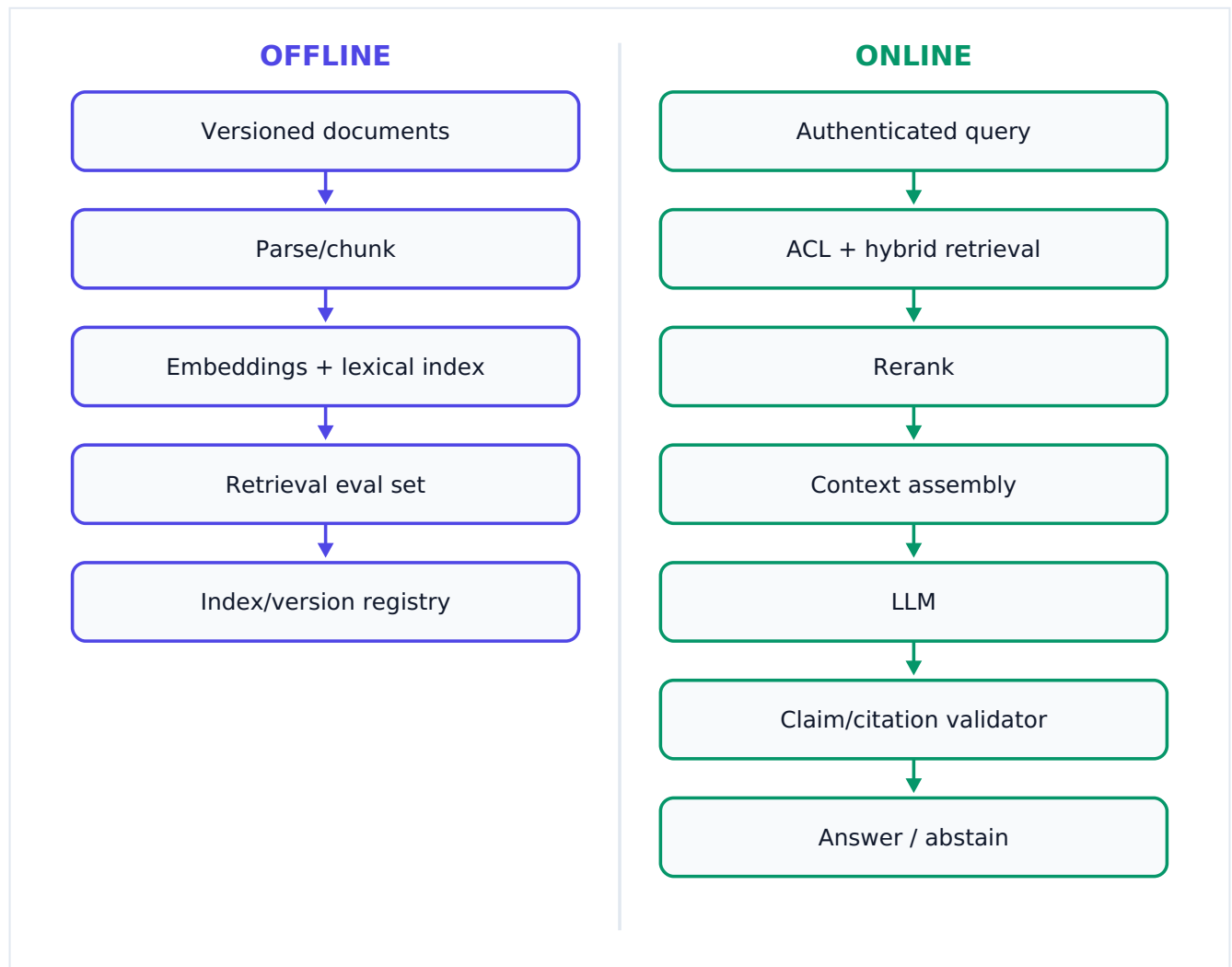


Figure: interview-level architecture. Explain arrows, failure boundaries and trade-offs.

SYSTEM DESIGN CASE2026 SIGNAL

Current 2026 ML system-design banks explicitly include GPU batching and LLM-serving capacity questions.

GPU inference batching service

Requirements

Serve interactive LLM/model requests while maximizing GPU utilization and keeping time-to-first-token and p95 latency within SLO.

Architecture

Gateway normalizes/authenticates requests, scheduler queues by model/shape/token profile, continuous batching packs compatible work, GPU workers execute prefill/decode, responses are demultiplexed/streamed. Autoscaling uses queue depth + GPU metrics; admission control protects the service under overload.

Success metrics

TTFT, tokens/s or items/s, p50/p95 completion latency, GPU utilization, queue time, OOM rate, fairness and cost/request.

Capacity / bottleneck

Batching improves throughput but waiting grows tail latency. KV-cache/activation memory caps active concurrency, especially for long prompts.

Failure modes

OOM, head-of-line blocking, unfair tenants, long-context monopolization, cold model load, retry storms, autoscaling lag.

Trade-offs

Throughput vs latency; batch size vs memory; fairness vs utilization; replicas vs cost; long context vs concurrency.

Likely follow-up

What admission-control rule would you implement during overload? How would static vs dynamic cache strategy affect capacity?

SYSTEM DESIGN CASE2026 SIGNAL

Feature-store questions are common at senior MLE level because data consistency is often more important than model architecture.

Shared online/offline feature platform

Requirements

Provide point-in-time-correct historical features for training and low-latency fresh features for many online models.

Architecture

Versioned feature definitions feed batch/stream computation. Historical/offline store supports time-travel joins; online store holds latest values. Feature service returns values with timestamps/version; lineage and validation monitor freshness and parity. Backfills use event-time semantics.

Success metrics

Online p95 latency, freshness SLA, offline-online parity, availability, backfill correctness, adoption, lineage coverage and cost.

Capacity / bottleneck

Maintaining one definition across batch and streaming while supporting late data and historical backfills without skew.

Failure modes

Training-serving skew, stale online values, schema changes, late events, duplicated feature logic, incorrect as-of joins.

Trade-offs

Central platform vs team autonomy; freshness vs cost; online latency vs consistency; generality vs specialized performance.

Likely follow-up

How do you guarantee point-in-time correctness during a historical backfill?

4. ML platforms, delayed labels and agentic systems

Platform questions reward precise contracts and observability. Agentic questions add a dynamic control loop but still require the same engineering discipline.

SYSTEM DESIGN CASE2026 SIGNAL

Current system-design question banks include experiment-tracking platform design for ML infrastructure roles.

ML experiment tracking and model registry platform

Requirements

Track thousands of training runs with parameters, metrics, artifacts, data/code lineage, comparisons, approval state and reproducibility.

Architecture

Client SDK emits run metadata to an API; metadata DB stores run identity, params and lineage; time-series/analytical store handles metrics; object store holds model/artifact blobs. Registry references immutable artifacts and promotion approvals. UI/search layer supports comparison; access control and retention apply.

Success metrics

Logging reliability, query latency, artifact durability, reproducibility rate, lineage coverage, storage cost and adoption.

Capacity / bottleneck

Metrics volume and artifact size have different storage patterns; separate metadata from blobs. Avoid mutable run identity.

Failure modes

Missing dataset/code versions, orphan artifacts, secret leakage, mutable tags, metric-schema drift, access-control mistakes.

Trade-offs

Rich lineage vs developer friction; centralized schemas vs flexibility; long retention vs cost.

Likely follow-up

How would you compare two runs when the data snapshot or metric definition changed?

SYSTEM DESIGN CASE2026 SIGNAL

LTV prediction appears in current MLE system-design banks because delayed labels force careful temporal evaluation.

Customer lifetime value (LTV) prediction

Requirements

Estimate 12-month customer value for acquisition/retention decisions even though full labels arrive slowly.

Architecture

Define observation window and outcome horizon. Build point-in-time customer/event snapshots, handle censored labels, train interpretable baseline then richer regression/survival/ranking model, output value with uncertainty where useful. Score in batch/nearline and feed decisions through a policy layer.

Success metrics

MAE/WAPE, rank/lift by value bucket, calibration, confidence interval coverage, campaign ROI and cohort stability.

Capacity / bottleneck

Long label horizon slows iteration; many recent users have censored outcomes. Backtests must respect observation/outcome windows.

Failure modes

Future-spend leakage, survivorship bias, channel shift, treatment effects from historical campaigns, inflation/currency issues.

Trade-offs

Long-horizon accuracy vs freshness; predictive LTV vs causal incremental value; interpretability vs flexibility.

Likely follow-up

How would you evaluate a model for users who have not yet accumulated 12 months of outcomes?

SYSTEM DESIGN CASE2026 SIGNAL

Agentic system-design cases increasingly appear in AI engineering interviews.

Agentic research workflow

Requirements

Research and draft a cited report using approved tools within a five-minute and fixed-cost budget.

Architecture

Goal/evidence checklist -> planner/state graph -> bounded search/tool proposals -> server-side validation/authorization -> evidence store -> coverage verifier -> synthesis -> claim/citation validation -> human approval or final output. Checkpoints persist state and side-effect receipts.

Success metrics

Task success, evidence coverage, unsupported claim rate, tool correctness, repeated-step rate, latency, cost and policy violations.

Capacity / bottleneck

Serial model/tool calls dominate latency; parallelize independent retrieval carefully but cap fan-out and centralize accepted evidence.

Failure modes

Prompt injection, endless search, duplicate side effects, fabricated citations, stale checkpoint, cross-tenant data leak.

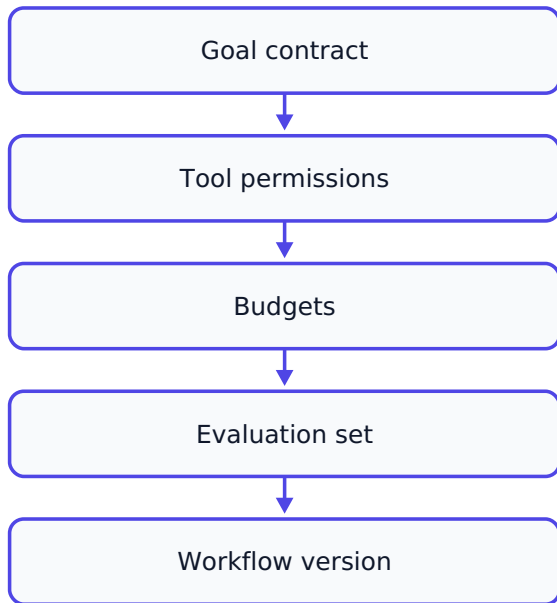
Trade-offs

Autonomy vs predictability; search depth vs cost; richer memory vs privacy; multi-agent parallelism vs coordination errors.

Likely follow-up

What exact stop conditions prevent looping, and what state must be checkpointed for safe resume?

CONTROL



RUNTIME

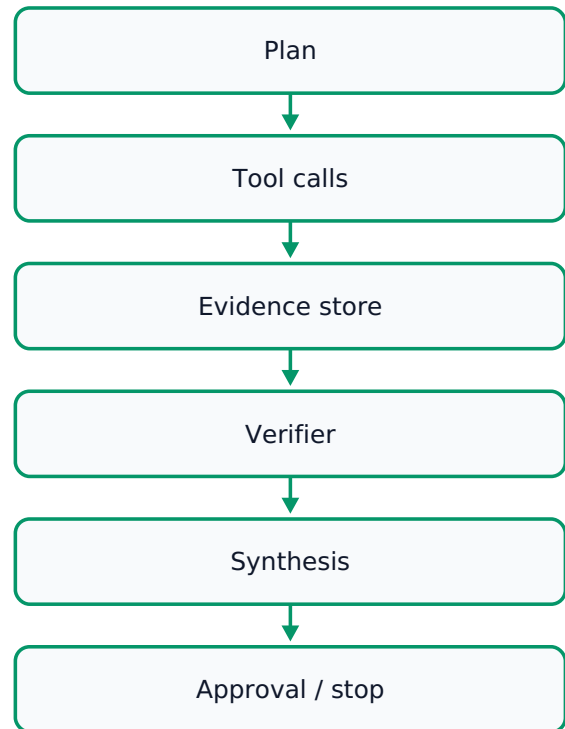


Figure: interview-level architecture. Explain arrows, failure boundaries and trade-offs.

Deep-dive ML/AI system design tutorial

This section gives a repeatable framework for architecting recommendation, ranking, fraud, RAG and GPU-serving systems under production constraints.

Study method Do not memorize definitions only. For each concept, explain the idea, work the example, name a failure mode, and state one trade-off. That pattern matches how stronger technical interviews are evaluated.

1. The seven-part ML system design answer

A reliable structure is: clarify product requirements and scale; frame the ML task; define offline and online metrics; design data/labels/features; choose model/training; design serving; design experimentation/monitoring/retraining.

This prevents jumping straight to architecture boxes without explaining what the system optimizes.

2. Metrics: business, online and offline

Business metrics capture product value; online metrics measure live user behavior; offline metrics estimate model quality from historical labels. Connect them explicitly. A recommender may optimize NDCG offline but care about watch time, diversity and retention online.

Use guardrails for safety, latency and ecosystem health.

3. Data and labels

Discuss sources, event schemas, label definition, delay, bias and leakage. Logs reflect the behavior of the existing system, so training data can be selectively observed. Negative examples may be ambiguous when an item was never shown.

Timestamp every feature relative to the prediction event and preserve lineage for reproducibility.

4. Candidate generation and ranking

Large recommendation/search systems rarely score every item with the most expensive model. Candidate generation retrieves hundreds or thousands cheaply; ranking applies richer features/models to a smaller set; optional reranking applies business, safety and diversity rules.

This multi-stage pattern balances recall, quality and latency.

5. Feature platform and training-serving consistency

A feature platform can centralize definitions, offline materialization and low-latency online lookup. The critical requirement is semantic consistency: the same feature definition should produce compatible values in training and serving.

Point-in-time correct offline joins prevent leakage. Freshness SLAs and default behavior matter online.

Deep dive continued

6. Training pipeline and model registry

Training pipelines validate data, build datasets, train/evaluate candidates, record lineage and register approved artifacts. A registry should link model version to code, data snapshot/feature definitions, metrics and deployment status.

Promotion should be policy-driven rather than “latest model always wins”.

7. Online serving

Serving design depends on latency and freshness. Batch predictions are cheap for slowly changing scores. Online inference handles request-specific features but needs reliable feature fetches, model loading, autoscaling and fallbacks.

Consider p50/p95/p99 latency, capacity, cacheability and failure behavior.

8. Feedback loops and exploration

Models influence what data gets collected. A recommender that only shows high-scoring items receives feedback mostly on those items, reinforcing its beliefs. Exploration, randomized traffic or counterfactual methods can reduce this bias.

Discuss diversity and long-term effects, not only short-term clicks.

9. Real-time fraud design

Fraud systems need low latency, fresh behavioral features and asymmetric costs. A common stack combines rules, streaming features and an ML score, with thresholds for approve/review/decline.

Labels can be delayed and adversaries adapt, so monitoring and human-review feedback are central.

10. RAG system design

A production RAG answer should include ingestion, parsing/chunking, lexical/vector indexes, authorization filters, hybrid retrieval, reranking, context packing, generation, citations/abstention and evaluation.

Treat prompt injection and tenant isolation as system concerns, not prompt wording only.

Deep dive continued

11. GPU inference design

LLM serving is constrained by accelerator memory and the prefill/decode workload. Use continuous batching, KV-cache management, quantization or parallelism depending on model size and SLOs.

Track TTFT, time per output token, throughput, queue depth and OOM events.

12. Experimentation, rollout and monitoring

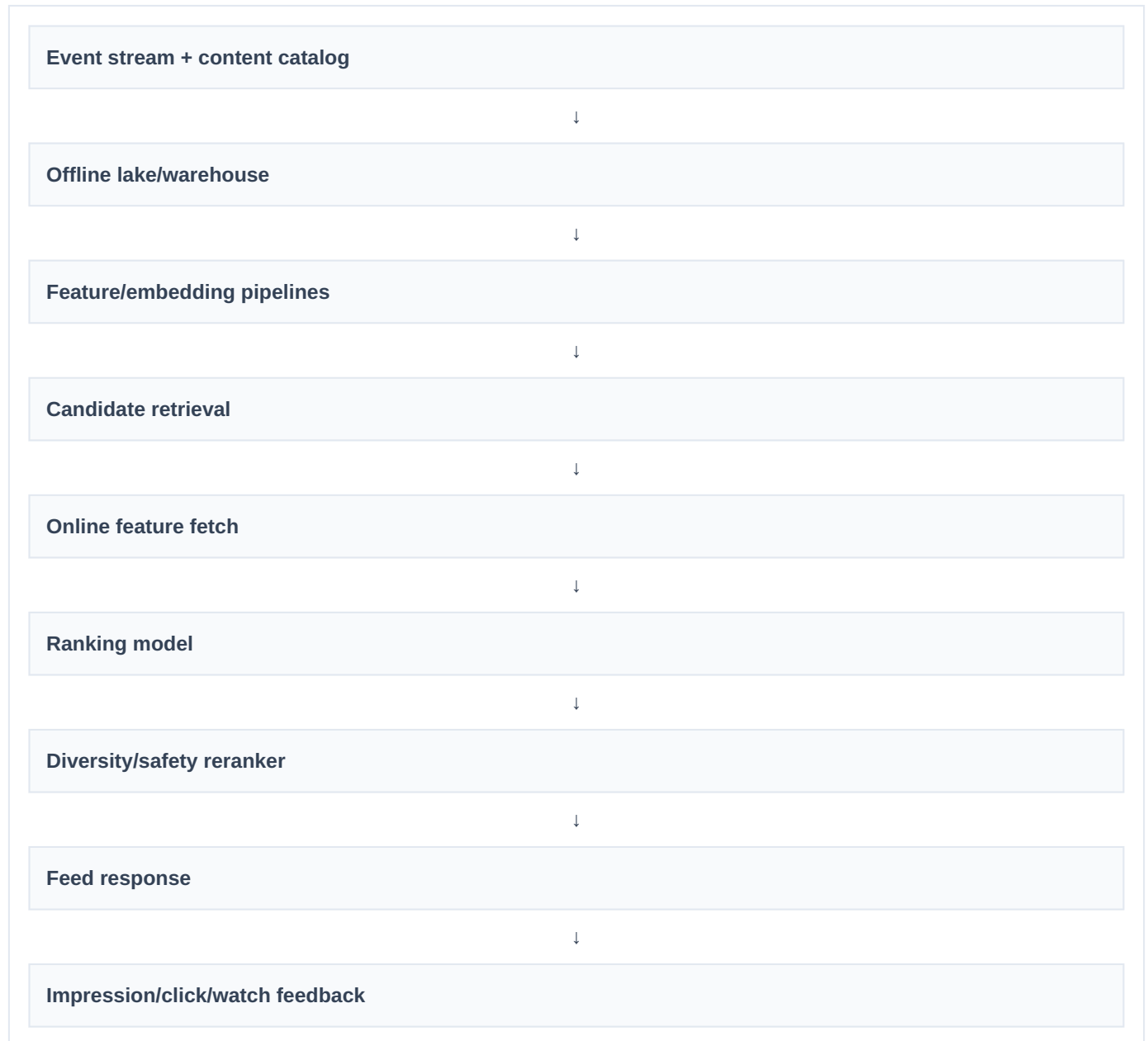
Use shadow traffic to validate infrastructure, canary releases to limit blast radius, and A/B tests for product impact. Monitor data health, model metrics, system metrics and business guardrails.

Have explicit rollback criteria. Retraining is a controlled response to evidence, not the default reaction to every drift alert.

Complete case walkthrough

Case study - short-video recommendation feed

Design: short-video recommendation feed at millions of requests per day.



Discuss user/content cold start, retrieval embeddings, freshness of watch-history features, two-stage ranking, latency budget, caching, diversity, safety, online A/B tests, delayed long-term retention metrics and feedback-loop mitigation.

Additional interview drill

Answer each question aloud in 60-90 seconds. Then add one example, one failure mode and one trade-off.

#	Question	Strong answer should mention
1	What do you clarify first?	product goal, user action, scale, latency, freshness, constraints
2	Offline vs online metrics?	proxy model quality vs live product impact
3	Why two-stage ranking?	cheap recall then expensive precision under latency budget
4	What is point-in-time correctness?	features use only data available before prediction
5	Batch vs online inference?	freshness/personalization vs simplicity/cost
6	What causes training-serving skew?	different code/data/freshness/defaults
7	How monitor recommender?	quality, latency, diversity, drift, business, feedback loops
8	How handle delayed labels?	proxy monitoring then backfill true performance
9	What belongs in model registry?	artifact, lineage, metrics, approvals, deployment state
10	How design RAG auth?	filter before context using identity/ACL metadata
11	What are GPU serving metrics?	TTFT, TPOT, throughput, queue, memory/OOM
12	How roll out safely?	offline gate, shadow, canary, A/B, rollback

Hands-on labs and worked exercises

These exercises turn the concepts into things you can calculate, code, debug or design. Work them before reading the expected reasoning.

Lab 1 - Requirements card

Goal Practice the first five minutes of system design.

Steps

1. For a feed recommender, write daily active users, QPS, p95 latency target, catalog size and freshness requirement.
2. Choose an online business metric and offline proxy.
3. List two safety/operational guardrails.

Expected reasoning / result

Numbers can be assumptions, but they should drive architecture choices.

Interview connection

Ask/declare scale instead of drawing infrastructure blindly.

Lab 2 - Two-stage recommender

Goal Design candidate generation plus ranking.

Steps

1. Generate candidates from collaborative/embedding/popularity sources.
2. Union/dedupe to hundreds.
3. Fetch fresh user/item features.
4. Rank with richer model.
5. Apply diversity/safety constraints.

Expected reasoning / result

Candidate generation optimizes recall under cost; ranking optimizes precision/value.

Interview connection

Discuss cold-start fallbacks and feedback loops.

Lab 3 - Feature-store parity test

GoalDetect training-serving skew.

Steps

1. For the same historical entity/timestamp, compute offline feature.
2. Replay equivalent online feature logic.
3. Compare values within tolerance across a sample.
4. Alert on systematic divergence.

Expected reasoning / result

Shared definitions and point-in-time retrieval reduce skew, but parity should still be tested.

Interview connection

Feature stores are valuable for semantics, not merely storage.

Lab 4 - Safe rollout plan

Goal Choose shadow, canary and A/B correctly.

Steps

1. Shadow new model to validate serving without user impact.
2. Canary small live traffic and monitor technical/quality guardrails.
3. Run A/B for causal product impact.
4. Define rollback thresholds and artifact version.

Expected reasoning / result

Each stage answers a different question: infrastructure, blast radius, then product value.

Interview connection

Do not call every traffic split an A/B test.

Lab 5 - RAG design under permissions

Goal Integrate security into architecture.

Steps

1. Attach tenant/ACL metadata during ingestion.
2. Authenticate user and build filters.
3. Run hybrid retrieval only over allowed scope.
4. Rerank/pack context with provenance.
5. Generate with citations and abstention.

Expected reasoning / result

Unauthorized text should never enter model context.

Interview connection

Security belongs in the data/retrieval plane as well as model prompting.

Last-minute revision sheet

Use this page in the final 30-60 minutes before an interview. Say each answer aloud in one or two sentences; if you cannot, revisit the relevant chapter.

Topic	One-line recall
Start	Clarify user decision, success metric, scale, latency and label availability before architecture.
Multi-stage ranking	Cheap high-recall candidate generation first; expensive high-precision ranking on shortlist.
Offline vs online	Offline builds data/model; online serves decisions under SLO with fallbacks and version logging.
Metrics	Use offline model metrics plus online product/business and reliability guardrails.
Point-in-time data	Features and labels must reflect only information available at decision time.
Failure handling	Every design needs fallback, rollback, timeout and degraded-mode behavior.
Security	Identity, authorization, data minimization and audit are architecture components.
Trade-offs	Explicitly choose between quality, latency, freshness, cost, complexity and privacy.

Research basis and further reading

This handbook is original LearnMLAcademy material. Public interview reports and official technical documentation were used to identify current topic emphasis and production practices. The explanations, examples, diagrams and code in this handbook are independently written.

Aced / Exponent - System Design Interview Prep (2026)

<https://www.tryexponent.com/blog/system-design-interview-guide>

Current system-design prep highlights cost, failure modes, operational judgment, AI/LLM infrastructure and agentic systems.

Educative - Machine Learning System Design Interview (2026)

<https://www.educative.io/blog/how-to-crack-machine-learning-system-design-interview>

Practical framework for scalable ML design, candidate funnels, data quality, ANN retrieval and latency budgeting.

Aced / Exponent - Meta Senior MLE Interview Experience (2026)

<https://www.tryexponent.com/experiences/meta-facebook-senior-machine-learning-engineer-interview-8719a7>

Candidate-reported Meta loop covering classic coding, AI-assisted debugging, behavioral questions and recommendation-system design with online/offline evaluation.

InterviewQuery - Google Data Scientist Interview Guide (2026)

<https://www.interviewquery.com/guides/google-data-scientist>

Recent candidate reports emphasize SQL window functions, A/B testing, product metrics, ML fundamentals and behavioral rounds.

Google Machine Learning Crash Course

Official ML learning material emphasizes practical exercises, production systems, testing and monitoring in addition to model theory.

Made With ML MLOps Course

Production-oriented curriculum covering data, modeling, testing, reproducibility, serving, CI/CD and monitoring.

Current 2026 interview-preparation resources

Public guides increasingly emphasize production trade-offs, system design, debugging and role-specific reasoning rather than definition-only question banks.

Keep learning at learnmlacademy.com

167 in-depth lessons • Python code examples • Visual diagrams • Projects • Interview preparation • Generative AI • LLM & RAG • Agentic AI • Always free